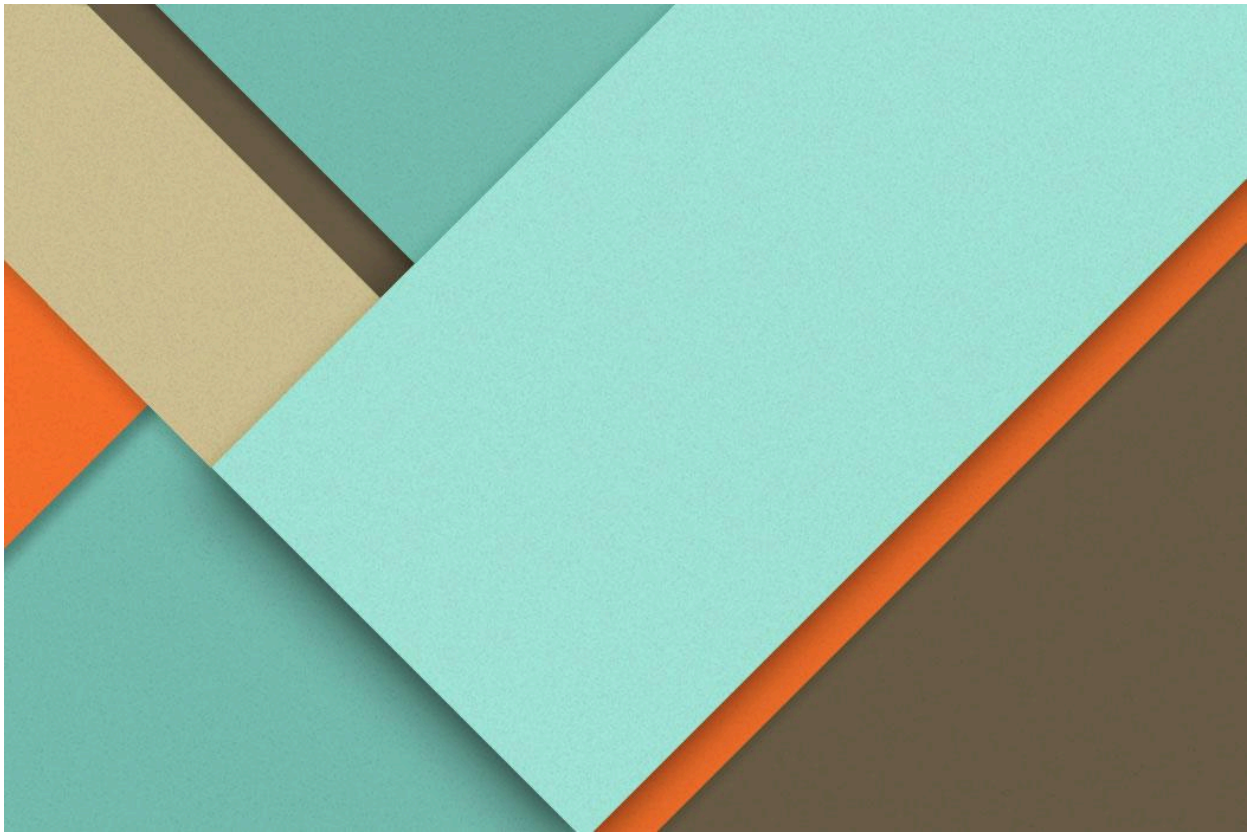


Processor Design Assignment Report

Made by Dhyaan Shah (25114032), Hitansh Gandhi (25114044), Nitesh Anand (25114065), Ritik Yadav (25114079) and Vansh Gupta (2511097)

GitHub submission link - https://github.com/nagi-17/Assignment-Processor-Design

Introduction and Objective

This report presents the design and evaluation of RISC201, a 32-bit RISC instruction set architecture, together with two pipelined processors that implement it: a 4-stage design (IF, ID, EX, merged MEM/WB) and a 6-stage design (IF, ID, EX, MA1, MA2, WB). The goal of the project was to understand computer architecture, not to write code, so the report concentrates on the reasoning behind each design decision: how the instruction set was shaped, why the pipeline stages were merged or split the way they were, how data and control hazards are resolved, and how the arithmetic units and control unit were built.

The main components are an integer ALU, a floating-point unit (FPU) following IEEE-754, and a vertically microprogrammed control unit whose control ROM generates the 37 control signals carried through the pipeline. Hazards follow the interlock-plus-forwarding scheme of Smruti Sarangi, using a data lock unit, a branch lock unit and a global hazard unit that handles traps and system instructions. The two pipelines are compared on instructions per cycle (IPC), total cycle count and stall frequency, for compute-heavy and memory-heavy workloads. A supporting software suite (two-pass assembler, disassembler, CLI simulator and debugger, stack macros, stack-boundary protection and an ASCII stack visualizer) is described at the end.

We chose a vertical microprogrammed control unit. A microprogrammed design trades a little speed for flexibility: the control logic lives in a ROM that can be changed without redesigning the datapath, which suited a project where the instruction set kept growing (system and safety instructions were added late).

Datapath Design

4-Stage Pipeline

The 4-stage processor has the stages IF, ID, EX and MEM/WB. Instruction fetch reads the instruction at the program counter. Instruction decode (ID) reads the register file, forms the immediate and branch target and looks up the control ROM. Execute (EX) runs the ALU or FPU and resolves branches. The final stage merges memory access with write-back: loads and stores access data memory and the result is written to the register file in the same stage.

6-Stage Pipeline

The 6-stage processor has IF, ID, EX, MA1, MA2 and WB. The memory stage is split in two. MA1 computes the memory address and checks it against the stack boundary registers (STACK_BASE and STACK_LIMIT), setting `stack_fault` in the MA1/MA2 latch on a violation. MA2 performs the actual data-memory access and hosts the global hazard unit, which handles traps and system instructions. WB writes the result into the register file.

Design Rationale: Which Stages to Merge and Split

Both pipelines start from the classic five stages (IF, ID, EX, MEM, WB). We compared the candidate changes before settling.

Why the 4-stage merges MEM and WB (and not ID/EX or IF/ID) :

- Merging ID and EX lengthens the cycle time, makes the design less logically clean (operand read and execute fight for the same stage), couples stalls (a stall in one forces the other), and removes both a forwarding source and a forwarding receiver. The effect on the branch penalty is unclear and has to be measured.
- Merging IF and ID is also worse than merging MEM and WB. MEM and IF are similar in latency and ID is worse than WB. Merging IF and ID doesn't do any benefit to the forwarding thing either.
- MEM and WB is the logically clean merge. The memory stage is idle for most instructions anyway, since only loads and stores use it, so combining it with the register write wastes little. It does not remove any forwarding path that matters, and it is the faster of the options.

Why the 6-stage splits MEM (and not EX) :

- Splitting EX is logically awkward (what would be divided?), and it creates the most painful dependency in the machine: ALU-to-ALU. ALU instructions are far more frequent than loads, so every dependent pair would stall, and a stall in one stage fans out to the others (stall fanout).
- The ALU is not the bottleneck, so splitting it buys little cycle-time improvement.
- Memory access is the long stage. Splitting it into address and boundary check (MA1) and data access (MA2) shortens the cycle and keeps ALU forwarding intact. The price is a longer load-use delay, which only affects loads.

Microprogrammed Machine Model

In a microprogrammed processor, every functional unit is wrapped with its own input and output registers, which removes the fixed interconnections of a hardwired design. The control unit then decides, by microinstruction, which registers are loaded and which unit operates. The internal registers are listed below.

Register	Bits	Function
pc	32	Program counter
ir	32	Instruction register
I	1	Immediate bit
rd, rs1, rs2	4 each	Destination and source register ids
immx	32	Immediate after processing modifiers
branchTarget	32	Computed as pc + offset
regSrc, regData, regVal	4, 32, 32	Register-file address, write data and read value
A, B	32	ALU operands
fA, fB	32	FPU operands
flags.E, flags.GT	1 each	Equality and greater-than flags
aluResult, fpuResult	32	Unit results
mar, mdr, ldResult	32	Memory address, data and loaded value

Pipeline Registers (6-Stage)

Every latch carries an `is_valid` bubble flag, essential for stalls and branch flushes, and `pc_next` where it may be needed later.

Latch	Contents
IF/ID	pc (32), ir (32), pc_next (32), is_valid (1)
ID/EX	37 control bits from the control ROM, pc_next (32), is_valid (1), rd (4), op1 (32), op2 (32), immx (32)
EX/MA1	exResult (32), is_Ld, is_St, is_Wb, is_Clear, rd (4), is_valid, st_data (32), is_Ecall, is_Ebreak, is_Eret, is_Wf
MA1/MA2	Same fields as EX/MA1, plus stack_fault (1)
MA2/WB	exResult (32), is_Ld, is_Wb, is_Clear, rd (4), is_valid (debug and statistics), read_data (32)

Two decisions are worth noting. First, `pc_next` is carried down the pipe so that function calls can store the return address and traps can save the resume address without a second 32-bit adder. Second, the ALU and FPU results share one `exResult` field, merged through an OR gate, since only one unit is active per instruction.

Performance Comparison: 4-Stage versus 6-Stage

Metrics

- **IPC** = instructions retired / total cycles.
- **Total cycle count** = cycles from the first fetch until the last instruction retires.
- **Stall frequency** = stall (bubble) cycles / total cycles, split by cause: data lock, branch, and multi-cycle EX.
- **Execution time** = total cycles × cycle time. The 6-stage should reach a shorter cycle time since the memory stage is split, while the 4-stage needs fewer cycles per dependency.

Workloads

We evaluate compute-heavy programs (long runs of ALU and FPU operations with few loads) and memory-heavy programs (frequent loads and stores, including load-use pairs), plus a control-heavy case with many branches. The expectation is that the 6-stage wins on compute-heavy code because of its shorter cycle, while load-use stalls erode its advantage on memory-heavy code.

Tests and Results

Test 1 :

```
mul r0, r1, r2      ; 20 times, operands never written
div r3, r1, r2      ; 8 times
```

Result :

```
======================================================================
                RISC201 PIPELINE EXECUTION COMPARISON
======================================================================
Metric                          4-Stage Pipeline    6-Stage Pipeline
----------------------------------------------------------------------
Instructions Retired            30                  30
Total Clock Cycles              215                 219
Clock Period (ns)               2.00                1.50
Total Execution Time            430.00 ns           328.50 ns
Stall Cycles                    182                 184
Stall Overhead                  84.7%               84.0%
CPI (Cycles/Instruction)        7.17                7.30
----------------------------------------------------------------------
  Analysis: 4-stage pipeline is 1.9% faster (fewer stall cycles across shorter hazard window).
  Wall-clock: 6-stage finishes sooner (328.50 ns vs 430.00 ns, 30.9% faster).
======================================================================
```

Test 2 :

```
ecall                ; 10 times, handler is a single eret
```

Result :

```
======================================================================
                RISC201 PIPELINE EXECUTION COMPARISON
======================================================================
Metric                          4-Stage Pipeline    6-Stage Pipeline
----------------------------------------------------------------------
Instructions Retired            400000              399998
Total Clock Cycles              1000000             1000000
Clock Period (ns)               2.00                1.50
Total Execution Time            2.00 ms             1.50 ms
Stall Cycles                    0                   0
Stall Overhead                  0.0%                0.0%
CPI (Cycles/Instruction)        2.50                2.50
----------------------------------------------------------------------
  Analysis: Both pipelines completed in the exact same number of clock cycles.
  Wall-clock: 6-stage finishes sooner (1.50 ms vs 2.00 ms, 33.3% faster).
======================================================================
```

Test 3 :

```
ld  r1, r4, 0       ; 10 loads, +64 each (new line, always a
miss)
add r2, r2, 1       ; one independent add after each load
```

Result :

```
======================================================================
                RISC201 PIPELINE EXECUTION COMPARISON
======================================================================
Metric                          4-Stage Pipeline    6-Stage Pipeline
----------------------------------------------------------------------
Instructions Retired            20                  20
Total Clock Cycles              1492                92
Clock Period (ns)               2.00                1.50
Total Execution Time            2.98 us             138.00 ns
Stall Cycles                    1469                67
Stall Overhead                  98.5%               72.8%
CPI (Cycles/Instruction)        74.60               4.60
----------------------------------------------------------------------
  Analysis: 6-stage pipeline completed with fewer cycles (1521.7% difference).
  Wall-clock: 6-stage finishes sooner (138.00 ns vs 2.98 us, 2062.3% faster).
======================================================================
```

Test 4a and 4b :

```
; unscheduled                    ; scheduled
loop: ld  r1, r4, 0              loop: ld  r1, r4, 0
      add r0, r0, r1  ; stall          add r4, r4, 1
      add r4, r4, 1                    cmp r4, r5
      cmp r4, r5                       add r0, r0, r1
      blt loop                         blt loop
```

Result 4a:

```
======================================================================
                RISC201 PIPELINE EXECUTION COMPARISON
======================================================================
Metric                        4-Stage Pipeline    6-Stage Pipeline
----------------------------------------------------------------------
Instructions Retired          7                   7
Total Clock Cycles            160                 26
Clock Period (ns)             2.00                1.50
Total Execution Time          320.00 ns           39.00 ns
Stall Cycles                  150                 14
Stall Overhead                93.8%               53.8%
CPI (Cycles/Instruction)      22.86               3.71
----------------------------------------------------------------------
  Analysis: 6-stage pipeline completed with fewer cycles (515.4% difference).
  Wall-clock: 6-stage finishes sooner (39.00 ns vs 320.00 ns, 720.5% faster).
======================================================================
```

Result 4b:

```
======================================================================
                RISC201 PIPELINE EXECUTION COMPARISON
======================================================================
Metric                        4-Stage Pipeline    6-Stage Pipeline
----------------------------------------------------------------------
Instructions Retired          7                   7
Total Clock Cycles            158                 22
Clock Period (ns)             2.00                1.50
Total Execution Time          316.00 ns           33.00 ns
Stall Cycles                  148                 10
Stall Overhead                93.7%               45.5%
CPI (Cycles/Instruction)      22.57               3.14
----------------------------------------------------------------------
  Analysis: 6-stage pipeline completed with fewer cycles (618.2% difference).
  Wall-clock: 6-stage finishes sooner (33.00 ns vs 316.00 ns, 857.6% faster).
======================================================================
```

Test 5 :

```
      mov r2, 10
      mov r0, 0
loop: add r0, r0, 1
      add r1, r1, r0
      cmp r0, r2
      blt loop
```

Result :

```
================================================================================
                    RISC201 PIPELINE EXECUTION COMPARISON
================================================================================
Metric                          4-Stage Pipeline    6-Stage Pipeline
--------------------------------------------------------------------------------
Instructions Retired            5                   5
Total Clock Cycles              12                  18
Clock Period (ns)               2.00                1.50
Total Execution Time            24.00 ns            27.00 ns
Stall Cycles                    4                   8
Stall Overhead                  33.3%               44.4%
CPI (Cycles/Instruction)        2.40                3.60
--------------------------------------------------------------------------------
  Analysis: 4-stage pipeline is 50.0% faster (fewer stall cycles across shorter hazard window).
  Wall-clock: 4-stage finishes sooner (24.00 ns vs 27.00 ns, 12.5% faster).
================================================================================
```

Analysis

Test 1: multi-cycle EX stalls are identical, so IPC is nearly equal and the 6-stage wins only on clock.

Test 2: this is the 4-stage's clearest win on cycles and IPC (83 vs 105). The 6-stage resolves traps one stage later.

Test 3: this is memory-bound, with both IPC values at about 0.02, so pipeline depth is irrelevant. The 6-stage's time lead is an artifact of fixing DRAM at 100 cycles (150 ns vs 200 ns). With DRAM fixed at 200 ns (about 133 cycles for the 6-stage), it takes about 1345 cycles = 2017.5 ns, equal to the 4-stage's 2026 ns.

Test 4: the load-use penalty is a software scheduling problem. Placing two independent instructions between ld and its use removes the 6-stage's extra stalls (4a vs 4b).

Test 5: branches resolve in EX in both, so stall counts match. Branches don't favour the 4-stage.

Instruction Set Architecture

Register Set

RISC201 uses a unified 32-bit register file of 16 registers, so every register id fits in 4 bits.

Register	Role
r0 to r3	General-purpose integer registers
r4 to r7	Scratch registers (caller-saved)
r8 to r11 (aliased f0 to f3)	General-purpose floating-point registers
r12	Zero register, hardwired to 0
r13	Frame pointer (top of the activation block)
r14	Stack pointer
r15	Return address

System registers (not directly accessible to the programmer): `pc`, `flags.E` (equal), `flags.GT` (greater than), `epc` (exception program counter) and `cause` (exception cause code).

Design decisions : Sharing one file between integer and float registers keeps the register file, read ports and forwarding network single, at the cost of fewer registers per type. A hardwired zero register (r12) gives a free constant and lets compare-style operations and moves reuse existing instructions. Separate caller-saved scratch registers simplify the calling convention, and dedicated FP, SP and RA registers make `call`, `ret` and stack macros simple to implement.

Instruction Formats

All instructions are 32 bits wide, with a 6-bit opcode whose top two bits select the instruction class (00 integer ALU, 01 FPU, 10 memory/system, 11 control flow). Bit 6 is the immediate flag `I`.

Format	Layout (bits)
R-type (3-address)	opcode(6) · I=0 (1) · rd(4) · rs1(4) · rs2(4) · padding(13)
R-type (2-address)	opcode(6) · I=0 (1) · padding(4) · rs1(4) · rs2(4) · padding(13)
R-type (1-address)	opcode(6) · I=0 (1) · rd(4) · padding(4) · padding(4) · padding(13)
I-type	opcode(6) · I=1 (1) · rd(4) · rs1(4) · imm(16) · padding(1)
F-type (only fmov)	opcode(6) · I=1 (1) · rd(4) · float imm(21)

Keeping `rd, rs1 and rs2` in fixed bit positions across formats means the register-file read can start before the opcode is fully decoded, which shortens the ID stage. The separate F-type exists because a float constant needs more immediate bits than the 16 available in the I-type.

Instruction Set

Integer ALU (class 00)

Instruction	Opcode	Operands
nop	000000	none
add, sub, mul, div, mod	000001 to 000101	rd, rs1, rs2/imm
cmp	000110	rs1, rs2/imm (sets flags.E, flags.GT)
and, or, xor	001000 to 001010	rd, rs1, rs2/imm
not	001011	rd, rs1
lsl, lsr, asr	001100 to 001110	rd, rs1, rs2/imm

Floating point (class 01)

Instruction	Opcode	Operands
fadd, fsub	010001, 010010	rd, rs1, rs2
fmul, fdiv	010011, 010100	rd, rs1, rs2
fcmp	010110	rs1, rs2 (updates the standard flags)
fmov	011000	rd, imm21

Memory and system (class 10)

Instruction	Opcode	Operands / purpose
ld	100000	rd, rs1, imm
st	100001	rs2, rs1, imm
movu	100010	rd, imm (loads the upper 16 bits)
movh	100011	rd, imm (loads the lower 16 bits)
csrr	101000	rd, csr_imm (read a system register)
csrw	101001	csr_imm, rs1 (write a system register)
chk	101100	rs1, rs2 (bounds check, traps if rs1 >= rs2)
clrz	101101	rd (securely zeroes a register)

Control flow (class 11)

Instruction	Opcode	Purpose
b, beq, bne, bgt, blt	110000 to 110100	Unconditional and conditional branches (flag based)
call, ret	111000, 111001	Function call and return (return address in r15)
ecall	111100	Environment call, triggers a trap to the OS/simulator

eret	111101	Exception return, restores PC from epc
ebreak	111110	Breakpoint, halts and drops into the CLI debugger
wfi	111111	Wait for interrupt, pauses execution until an external event

`push` and `pop` are assembler preprocessor macros, and `mov` is implemented as an `add` with the zero register or an immediate.

Why do the system instructions exist ?
General registers cannot talk to system registers, so `csrr` and `csrw` are the controlled doorway. `ecall`, `eret`, `ebreak` and `wfi` give the processor a software-triggered trap, a way back from it, a debugger hook and a low-power wait. `chk` and `clrz` are safety instructions: `chk` raises an out-of-bounds exception when `rs1 >= rs2`, and `clrz` wipes a register so secrets do not linger.

Hazard Handling

Hazard handling follows Smruti Sarangi's interlock-plus-forwarding scheme. A Data Lock Unit and a Branch Lock Unit sit in the ID stage. Their outputs are ORed into a single `isStall` signal, and a global hazard unit in the last stages handles traps and system instructions.

Data Hazards

A read-after-write (RAW) hazard arises when an instruction needs a register that an older, still in-flight instruction has not yet written. Two mechanisms resolve it.

- **Locking (interlock).** When the value is genuinely not available yet (a load whose data is still in memory), the Data Lock Unit compares the source registers of the instruction in ID against the destination registers in flight, and stalls: the PC and IF/ID hold, and a bubble enters ID/EX.

Dependency	4-stage	6-stage
ALU followed by dependent ALU	0 stall (forwarded)	0 stall (forwarded)
Load followed by dependent instruction	about 1 stall	about 2 stalls

The load-use figures are derived analytically from where load data becomes available (end of the merged stage versus end of MA2) and are confirmed against simulator output in Section 8.

Control Hazards: Branch Lock and Branch Handling

Branches are resolved in EX. Until the outcome is known, the Branch Lock Unit prevents the instructions behind the branch from taking effect, and on a taken branch the instructions in IF and ID are squashed by clearing `is_valid`. `call`, `b` and `ret` are always taken and `ret` takes its target from the return-address register. Conditional branches use `flags.E` and `flags.GT`, set by `cmp` or `fcmp`. [TO FILL: exact branch policy (always not-taken / lock until resolved), measured taken-branch penalty in each pipeline]

Traps, Exceptions and System Instructions

The instruction fetch stage has no latch in front of it. Its control inputs arrive as feedback wires from later stages, and the next PC is chosen by a multiplexer with this priority (oldest instruction wins):

1. **Trap or exception:** PC becomes the trap vector, EPC is saved. Raised by `ecall`, `chk`, a stack fault or an illegal instruction.
2. **eret:** PC becomes EPC.
3. **Branch taken (from EX):** PC becomes the branch target.
4. **Stall (from the lock units):** PC holds.
5. **Normal:** PC becomes pc + 1.

Because the assembler outputs machine code as an array of instructions, the simulator's PC is a word index that advances by one, which corresponds to pc + 4 in a byte-addressed machine. Trap, eret, `ebreak` and `wfi` are resolved in the global hazard unit (MA2 in the 6-stage, the merged MEM/WB stage in the 4-stage), which flushes all younger instructions. On `ecall`, EPC holds the `pc_next` of the call so that `eret` returns past it, while for faults EPC holds the faulting PC. Resolving eret in the last stage also avoids a race with a `csrw epc` still in flight. Since the resolution stage is later in the 6-stage, a trap flushes up to 4 younger instructions there versus 3 in the 4-stage.

Arithmetic Logic Unit

The ALU executes the 14 integer operations of class 00: add, sub, mul, div, mod, cmp, and, or, xor, not, lsl, lsr and asr. `cmp` performs a subtraction-style comparison that updates `flags.E` and `flags.GT` without writing a register. Operand B is either `rs2` or the sign-processed immediate (`immx`), selected by the `I` bit.

Operations	Hardware	Why it was chosen
add, sub, cmp	Kogge-Stone adder	Parallel-prefix adder with logarithmic depth. sub and cmp reuse it by inverting operand B and setting the carry-in.
lsl, lsr, asr	Barrel shifter	Any shift amount in one pass through 5 multiplexer levels, instead of up to 31 serial shifts.
mul	Radix-4 Booth encoder, Wallace tree of carry-save adders, Kogge-Stone final adder	Halves the partial products and reduces them without carry propagation.
div, mod	Radix-4 divider	Produces two quotient bits per iteration.
and, or, xor, not	Bitwise gate array	A single gate level, no carry chain.

Kogge-Stone adder : A ripple-carry adder needs a delay proportional to the word length, which would sit directly on the critical path of EX. A Kogge-Stone adder computes generate and propagate signals for every bit and then combines them in a parallel-prefix network of log2(32) = 5 levels, so the carry into every bit is ready after the same small number of gate delays. Its limited fan-out keeps each level fast. The cost is a large number of cells and wires, compared with sparser prefix designs such as Brent-Kung, which use fewer cells but more levels. We chose speed because the adder is used by `add`, `sub`, `cmp`, address calculation for `ld` and `st`, and the final step of `mul`, so it is the most heavily used block in the datapath.

Barrel shifter : The shifter has 5 stages of 2:1 multiplexers, shifting by 1, 2, 4, 8 and 16 positions, so any amount from 0 to 31 takes the same fixed delay. One structure covers all three shifts: `lsl` and `lsr` fill with zeros, and `asr` fills with the sign bit to preserve the sign of a two's-complement value.

Booth-Wallace multiplier : Multiplication has three steps. Radix-4 Booth encoding recodes the multiplier so that a 32-bit operand needs only 16 partial products instead of 32, and it handles signed operands directly. A Wallace tree of carry-save adders (3:2 compressors) then adds those partial products. A carry-save adder does not propagate carries between bit positions, so each level has the delay of one full adder regardless of word length, and the tree reduces the rows to two in a handful of levels. Finally, the two remaining rows are added with the Kogge-Stone adder to produce the result written to `rd`.

Radix-4 divider : A radix-2 divider produces one quotient bit per step and needs 32 iterations for a 32-bit operand. A radix-4 divider produces two quotient bits per iteration, so it needs 16. Each iteration is more complex (selecting the quotient digit and forming multiples of the divisor), but the iteration count is halved, which matters because division is by far the slowest integer operation.

Note - In ALU and FPU both, multiply takes 4 cycles however in ALU, division and modulo takes 16 cycles while in FPU, division takes 6 cycles.

Floating-Point Unit

The FPU implements IEEE-754 binary32 (1 sign, 8 exponent, 23 mantissa bits) on the float registers f0 to f3 (aliased to r8 to r11). It has five operations (`fadd`, `fsub`, `fmul`, `fdiv`, `fcmp`), each in its own unit. All units evaluate in parallel and a 4:1 result mux picks one. `fmov` loads a 21-bit float immediate through the F-type format. The common design choices are:

- **Denormals flushed to zero (FTZ) at unpack.** Supporting denormals would need extra normalisation hardware in every unit for numbers that rarely occur. Flushing them keeps each datapath simple.
- **Round-to-nearest-even** using guard, round and sticky bits, applied by one shared `roundAndPack` block used by add, multiply and divide. RNE is the IEEE default and is unbiased.
- **Special cases handled up front.** NaN, infinity and zero are detected at unpack, and each unit has a special-case detector that exits early. Any NaN result is the canonical `0x7FC00000`.
- **Reuse of arithmetic blocks.** The same Kogge-Stone adder, barrel shifter, carry-save adder and Booth-Wallace multiplier designs as in the ALU are used throughout, so only a small set of components has to be verified.

FADD / FSUB

FSUB is FADD with the sign of B flipped. The datapath is 27 bits (24-bit mantissa plus 3 GRS bits). The operands are compared and swapped so that A has the larger magnitude, so the mantissa subtraction never goes negative. B is then shifted right by the exponent difference, with shifted-out bits merged into the sticky bit. A 27-bit Kogge-Stone adder adds or subtracts. After a same-sign add, at most a 1-bit right shift normalises. After a subtract, a leading-zero counter and a barrel shifter renormalise (this handles cancellation). Then the result is rounded and packed.

FMUL

FSUB is FADD with the sign of B flipped. The datapath is 27 bits (24-bit mantissa plus 3 GRS bits). The operands are compared and swapped so that A has the larger magnitude, so the mantissa subtraction never goes negative. B is then shifted right by the exponent difference, with shifted-out bits merged into the sticky bit. A 27-bit Kogge-Stone adder adds or subtracts. After a same-sign add, at most a 1-bit right shift normalises. After a subtract, a leading-zero counter and a barrel shifter renormalise (this handles cancellation). Then the result is rounded and packed.

FDIV

We chose Newton-Raphson over a digit-recurrence divider because it reuses the multiplier. A 16-entry seed ROM, indexed by 4 leading divisor bits, gives an initial reciprocal. Each of the 3 iterations refines it as $X = X \cdot (2 - D \cdot X)$ in Q2.30 fixed point, and the quotient is $N \times (1/D)$. Each iteration roughly doubles the number of correct bits, so 3 iterations from a 4-bit seed are enough for a 24-bit mantissa. The exponent is `expA - expB + 127`.

FCMP and Flags

FCMP produces flags only, with no register result. A priority encoder checks in order: NaN (unordered), both zero (equal), different signs, exponent difference, then mantissa difference. For arithmetic results, a status flag generator derives Z, N, V and NaN from the result, and a 2:1 flag-source mux selects between these and the FCMP flags.

Operation	Core hardware	Cycles in EX
fadd, fsub	27-bit Kogge-Stone, leading-zero counter, barrel shifters	1
fmul	4× 16×16 Booth-Wallace-Kogge	4
fdiv	Newton-Raphson, 3 iterations (7 serial multiplies)	6
fcmp	Two Kogge-Stone subtractors and a priority encoder	1

Vertical Microprogrammed Control Unit

Why Microprogramming and Why Vertical

A microprogrammed control unit stores control behaviour in a control ROM instead of hardwired gates. Its advantages are flexibility (new instructions are new ROM entries) and easy debugging, and its cost is a ROM lookup in the critical path. We implement a vertical microprogram: microinstructions hold encoded fields that a small decoder expands into control signals. Compared with horizontal microcode, the ROM is much narrower (fewer bits per word and less memory), at the cost of a decoding step and fewer operations in parallel. Since the pipeline already overlaps instructions, the lost parallelism inside a single microinstruction costs little.

Control ROM Organisation

In the ID stage the opcode (and the `I` bit) addresses the control ROM, and the result is expanded into the **37 control signals** carried in the ID/EX latch, such as register write enable, memory read and write, ALU/FPU select, branch type, `is_Ld`, `is_St`, `is_Wb`, `is_Clear` and the system-call flags `is_Ecall`, `is_Ebreak`, `is_Eret` and `is_Wf`. The simulator displays the microPC step by step.

Since we are using a control ROM based microprogrammed unit, the concept of microprogram counter does not align very well with our ideas, hence we show the micro-program counter status as **unavailable**. However, this can be taken into account if we improve the implementation by including a microprogram execution unit in each of the pipeline latches.

Software Environment and Simulation Framework

The software suite exists to make the architecture testable. It is summarised briefly here, and details are in the repository.

- **Two-pass assembler:** Pass one records labels and expands the `push` and `pop` macros, pass two encodes every instruction into 32-bit binary or hexadecimal strings using the formats of Section 2.2.
- **Disassembler:** Parses binary or hex words and rebuilds the assembly text, which is used to check the assembler.
- **CLI simulator:** An interactive command-line interface that models the register file, the instruction pipeline and the microPC, and runs either pipeline.
- **Debugger:** Step-by-step execution, register and pipeline inspection, and breakpoints, entered through `ebreak` or CLI commands.
- **Stack preprocessor macros:** `push` and `pop` expand into full-descending stack operations on the stack pointer r14.
- **Stack boundary safety:** Addresses are checked against STACK_BASE and STACK_LIMIT in MA1 (6-stage), and a runtime exception framework raises a fault on stack corruption. The `chk` instruction offers the same protection to programs.
- **Stack visualizer:** An ASCII renderer that draws the stack frames during execution.
- **Microprogrammed control and ALU in the simulator:** The simulator runs RISC201 through the control ROM rather than hardwired logic, with an optimised ALU class driving all arithmetic, shift and logical operations.

Overall findings

- The 6-stage wins on wall-clock for compute-heavy and multi-cycle code (Test 2) and for code scheduled to avoid load-use pairs (Test 5), because the shorter clock outweighs its small cycle-count penalty.
- The 4-stage wins when loads are used immediately (Test 1), when traps are frequent (Test 3) and for short branch-heavy loops (Test 6), because it has fewer cycles per dependency and a shallower flush.
- ALU-to-ALU dependencies are forwarded with zero stalls in both designs, which confirms the decision to split MEM instead of EX.
- Memory-bound code (Test 4) is limited by the memory system, not by pipeline depth.

Limitations

Most programs are short, so fill and drain overhead inflates the stall percentages. Results also depend on the simulator's cache and DRAM model.

Conclusion

The project as a team task helped us explore and elevate our understanding of the practical application of ISA and microprogrammed control unit to a level that far exceeds a simple book read. We explored the various alternatives that were available at each crossroad and tried to implement what suited our needs and resources the best. We continue to explore the unimplemented possibilities and learn as much as we can and further learn optimisations that we couldn't figure out in this project.